Notes on Data Encryption

Riccardo Lo Iacono

March 17, 2026

Document is WIP, typos could be found

Contents.

1 Introduction **1**
1.1 About these notes 1

2 Mathematical background **2**
2.1 Some abstract algebra 2
2.2 Notions of projective geometry . . 4

3 Public-key cryptography **6**
3.1 The public-key exchange protocol . 6
3.2 The RSA algorithm 6
3.3 The ElGamal algorithm 7

4 Digital signature **10**
4.1 RSA digital signature 10

4.2 ElGamal digital signature 10

5 Integers factorization **12**

5.1 Pollard's $p-1$ algorithm 12

5.2 Pollard's ρ algorithm 13

5.3 Lenstra factorization 14

5.4 Extended Euclidean algorithm . . . 14

5.5 The Square and multiply algorithm 15

References **16**

Acronyms **17**

- 1 - Introduction.

Data encryption dials with how informations, see [4] for the definition, can be transmited securely through a channel.

More in details, if m is a message we want to send through a channel, first we have to encode it in a form that allows it to be transmited; then we want that such message is meaningless to those who have no right to read it. On the other hand, we want to be able to retrieve the message once this is recived.

To achive the above two disciplines were found: Cryptography and Code theory; the former focuses on how to make data secure, the latter provides methods to transmit data through a channel.

- 1.1 - About these notes.

These notes will focus on both Cryptography and Code theory, or at least, some parts of them.

More precisely, we begin by introducing some fundamental concepts of abstract algebra and projective geometry, we then discuss what public key encryption is focusing on the RSA and the ElGamal algorithm; we continue with describing some algorithms to factor integers, and conclude with several types of codes.

The following notation will be used throughout the document.

- $\mathcal{A}, \mathcal{B}$ and $\mathcal{E}$ will indicate, respectively, the sender, the reciever and an unauthorized individual attempting to capt our message.
- $\mathcal{C}$ will denote codes.
- Words in a code $\mathcal{C}$ will be represented by $\mathbf{w}$, with the occasional use of subscripts.

Any other symbol will be explained beforehand.

- 2 - Mathematical background.

In this section we provide the reader with some notions of abstract algebra; we recall what Greatest Common Divisor (GCD) and Least Common Multiple (LCM) are, we discuss congruence modulo n, and present groups and rings, and discuss some well know results on them.

Also, some broad concepts of projective geometry will be given; we focus on what projective space is, what the sum of projective points looks like, and to conclude, we give consider elliptic curves.

- 2.1 - Some abstract algebra.

In what follows, we assume the reader to have a basic understanding of number sets and equivalence relations.

Definition (Divisibility) Let $a, b \in \mathbb{Z}$. We say that a divides b, and write $a|b$, if

$$\exists q \in \mathbb{Z} \text{ s.t. } b = qa.$$

Otherwise we say that a does not divide b, and write $a \nmid b$.

Divisibility allows us to define two important concepts: LCM and GCD.

Definition (LCM) Let $a, b \in \mathbb{Z}$. Let $m \in \mathbb{Z}$. We say that m is the LCM for a and b if, the following holds:

1. $a|m$ and $b|m$, and
2. If $\exists m' \in \mathbb{Z}$ such that $a|m'$ and $b|m'$, then $m|m'$.

Definition (GCD) Let $a, b \in \mathbb{Z}$. Let $d \in \mathbb{Z}$. We say that d is the GCD for a and b if, the following holds:

1. $d|a$ and $d|b$, and
2. If $\exists d' \in \mathbb{Z}$ such that $d'|a$ and $d'|b$, then $d'|d$.

The computation of the GCD is done by the euclidean algorithm; we present an efficient inpementation of it in *Section* **??**.

ToDo. Add label in section 4.

Definition (Congruence modulo n) Let $a, b \in \mathbb{Z}$, and let $n \in \mathbb{Z}$ be a fixed integer. We say that a is congruent b modulo n, and write $a \equiv b \pmod{n}$ if and only if

$$\exists k \in \mathbb{Z} \text{ s.t. } a = kn + b.$$

- 2.1.1 - A brief introduction to groups.

Definition (Group) Let G be a set. Assume $\cdot : G \to G$ to be some function. We say that the pair $(G, \cdot)$, often simply as G, is a group if the following properties hold:

- $\forall a, b, c \in G, a \cdot (b \cdot c) = (a \cdot b) \cdot c$;
- $\exists e \in G \ \forall g \in G$ s.t. $g \cdot e = e \cdot g = g$;
- $\forall g \in G \ \exists g^{-1} \in G$ s.t. $g \cdot g^{-1} = g^{-1} \cdot g = e$.

If it also holds that $\forall a, b \in G, a \cdot b = b \cdot a$, we say that G is an abelian (or commutative) group.

We consider finite groups: that is, group with finitely many elements. Precisely, we are interested on the following class of groups.

Definition (Cyclic group) Let G be a group, and let $g \in G$. Denote by g^k the application of $\cdot$ on g, k times. Denote by $\langle g \rangle$ the set of all powers of g. We say that G is cyclic if $\langle g \rangle = G$.

- 2.1.2 - A brief introduction to rings.

Definition (Ring) Let R be a a set, and let $+, \cdot$ be two binary operations on R. We say that the triplet $(R, +, \cdot)$, often as R, is a ring if and only if the following conditions hold.

1. For all $a, b \in R, a + b = b + a$.
2. $a + (b + c) = (a + b) + c, \forall a, b, c, \in R$.
3. There exist a neutral element for $+$; that is, $\exists 0 \in R$ s.t. $a+0 = a, \forall a \in R$.
4. Each element has an additive oppposite, i.e., $\forall a \in R, \exists -a \in R$ such that $a + (-a) = (-a) + a = 0$.
5. $\cdot$ is associative; that is, $a(bc) = (ab)c, \forall a, b, c \in R$.
6. For all $a, b, c \in R, a(b + c) = ab + ac$.

Definition (Unital ring) Let $(R, +, \cdot)$ be a ring. We say that R is a unital ring, or a ring with unity, if there exists $1 \in R$ such that $1 \neq 0$ and $1a = a1 = a, \forall a \in R$.

Definition (Abelian ring) Let $(R, +, \cdot)$ be a ring. If for all $a, b \in R$ it holds that $ab = ba$; R is called commutative, or abelian, ring.

Definition (Integral domain) Let $(R, +, \cdot)$ be a commutative ring. If, for all $a, b \in R$ such that $ab = 0$, either $a = 0$ or $b = 0$; we say that R is an integral domain.

Definition (Division ring) Let $(R, +, \cdot)$ be a unital ring. We say that R is a division ring if, for all $a \in R$ exists a unique $a^{-1} \in R$ such that $aa^{-1} = a^{-1}a = 1$.

Definition. Let $(R, +, \cdot)$ be a ring. We say that R is a field, if and only if R is a commutative division ring.

In the following we denote fields with $\mathbb{F}$, while Galois fields, i.e., fields with finitely many elements, with $\mathrm{GF}(q)$ where q is the number of elements in the fields.

In *Section* **??** we discuss several codes, most of which make use of polynomial. In this context, an issue may occur: the chosen polynomial has no solution within the field in use. To solve it, we make use of field extensions, which are formally defined below.

Definition. Let $\mathbb{F}$ be a field. Denote by $\mathbb{F}[x]$ the set of polynomials with coefficients in $\mathbb{F}$[1]. Let $f(x) \in \mathbb{F}[x]$ be a irreducible polynomial, i.e., a polynomial with no solution[2]. We call field extension a field $\mathbb{F}'[x]$ in which we impose the existence of a solution for $f(x)$. That is,

$$\mathbb{F}'[x] = \{a + bi \mid a, b \in \mathbb{F} \text{ and } i \text{ root for } f(x)\}.$$

- 2.2 - Notions of projective geometry.

In what follows we assume that notions such as points, lines, planes in the geometrical sense, in addition to the notion of vector spaces are known to the reader.

Definition (Ray) Let $\mathbb{F}$ be a field, and let $\mathcal{V}$ be a vector space over $\mathbb{F}$. Let $\mathbf{v} \in \mathcal{V}$ be non-zero. We call the set

$$[\mathbf{v}] = \{k\mathbf{v} \mid k \in \mathbb{F}^*\}$$

a ray.

Definition (Projective plane) Let $\mathbb{F}$ be a field and let $\mathcal{V}$ be a vector space over F. Let $\mathrm{Dim}(\mathcal{V}) = 3$. We define the projective plane $\mathbb{P}^2$ as the set of rays of V. An element $X \in \mathbb{P}^2$ is called a point.

[1] It can be proved that $\mathbb{F}[x]$ is also a field.

[2] To be more precise a polynomial with no solution is garanteed to be irreducible if its degree is at most 3.

In our discussion we usually consider projective points in *homogeneous* coordinates; that is, if $X = [x : y : z]$ is a projective point, at least one of x, y, z is non-zero.

The relation with ordinary plane geometry is done as follow:

- Any point $(x, y) \in \mathbb{R}^2$ becomes a projective point $[x : y : 1]$; and
- Any point $[x : y : 0] \in \mathbb{P}^2$, which do not belong to Euclidean geometry, are called point at infinity, and we denote them as $\mathcal{O}$.

We now present *elliptic curves* which are the main reason we discuss projective geometry.

Definition (Elliptic curves) Let $\mathcal{V}$ be a vector space over a field F. An elliptic curve $\mathcal{C}$ is the set of points of $\mathbb{P}^2$, that are solution to an equation of the form

$$y^2 = x^3 + ax + b$$

with $a, b \in \mathbb{F}$.

In our treating, we consider non-singular elliptic curves; that is, such that the discriminant is non zero, i.e., $\Delta = -16(4a^3 + 27b \neq 0)$.

In *Section* **??** we treat the ElGamal algorithm by means of elliptic curves; in this context given $\mathcal{C}$ an elliptic curve, we are required to compute the sum of two point $P, Q \in \mathcal{C}$, which is non trivial. In what remain of this section, we shall present how to do such addition.

Let $\mathcal{C}$ be an ellitptic over the field $\mathbb{F}$ and let $P = (x_1, y_1), Q = (x_2, y_2) \in \mathcal{C}$. Then, the following cases hold:

- The points are opposite of one another, then $P + Q = P + \mathcal{O} = P$.
- The two points are distinct; then we consider the line that passes through them. That is, we compute

  $$x_3 = m^2 - x_1 - x_2 \qquad y_3 = m(x_1 - x_3) - y_1$$

  where $m = {}^{(y_2 - y_1)}/_{(x_2 - x_1)}$ is the slope.
- The two point coincide; then we consider the tangent line. That is, we compute

  $$m = \frac{3x_1^2 + a}{2y_1}$$

  and x_3, y_3 as before.

- 3 - Public-key cryptography.

Modern days cryptography is based on a set of protocols based on the concept of *public-key* exchange. We proceed as follow: the idea of public-key cryptography is defined first, then some protocols will be presented. For the latter we discuss both the algorithms and the mathematical hard problem these are based on.

- 3.1 - The public-key exchange protocol.

Proposed for the first time in [1], it was a breaktrhough in cryptography. The idea is that of crypting the message as some discrete value, by means of a one-way function; that is, a function that is easy to compute, but whose inverse is hard to compute.

The whole cryptosystem relies on the assumption that one-way functions do exists; through the years several have been proposed and are used in schemes we discuss in the following, yet these are not garanteed to be one-way[1].

- 3.2 - The RSA algorithm.

The RSA algorithm, from the name of its inventors: Rivest, Shamir and Adleman (see [3] for major details); is a public-key algorithm based on the hard problem of integers factorization.

Formally speaking, the Integers Factorization Problem (IFP) is defined as follow: given a positive integer n, find its prime factors; that is, find $p_1, p_2, \ldots, p_k$ such that $n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$, with $\alpha_i \geq 1$.

- 3.2.1 - The algorithm.

Let m be the message to transmit; to cypher it by the RSA protocol do as follow: begin by choosing two odd distinc prime numbers p, q randomly, such that their bitlength is approximately the same. Compute $n = pq$ and $\lambda = (p-1)(q-1)$. Let $\mathcal{B}$ choose an integer e such that $\mathrm{GCD}(e, \lambda) = 1$; the pair (n, e) will be $\mathcal{B}$'s *public key*. Let d be the integer less then λ, such that $ed \equiv 1 \pmod{\lambda}$; this will be $\mathcal{B}$'s *private key*.

To encode the message $\mathcal{A}$ computes

$$c \equiv m^e \pmod{n}$$

and sends c to $\mathcal{B}$. Upon reciving the chypertext, $\mathcal{B}$ can easily decode m as

$$m \equiv c^d \pmod{n}. \tag{1}$$

[1] If one proves the existence of one-way functions, at the same time would solve the $\mathcal{P} = \mathcal{N}P$ problem.

To see why *Equation* (1) holds observe the following: since $ed \equiv 1 \pmod{\lambda}$, there exists k such that $ed = 1 + k\lambda$.

Now, we have that:

- If $\mathrm{GCD}(m, p) = 1$ by Fermat's little theorem
$$m^{p-1} \equiv 1 \pmod{p}.$$
Hence, raising both sides by $k(q-1)$ and multipling by m yields
$$m^{1+k\lambda} \equiv m \pmod{p} \implies m^{ed} \equiv m \pmod{p}.$$
- If $\mathrm{GCD}(m, p) = p$ the above still holds, as both sides will be congruent to 0.

Applying the same reasoning for q, since p and q are distinc, we conclude that
$$c^d \equiv m \pmod{n}.$$

Let us briefly overview why $\mathcal{E}$ cannot, at least no so easily, dechypher the message sent by $\mathcal{A}$. By the protocol both n, e are know to everyone, but the values of λ and d are not as these depend on p and q. Hence, the only choice left to $\mathcal{E}$ is to factor n; which we know is hard to do.

Example

Let $p = 37, q = 39$. We have $n = 1443$ and $\lambda = 1368$. Say $\mathcal{B}$ chooses $e = 263$ for is public key. At this point the pair $(n, e) = (1443, 263)$ is know to everybody. $\mathcal{A}$ converts is message into an integer $0 < m < n$, say $m = 1029$ and computes
$$c \equiv m^e \pmod{n} \implies 1320 \equiv 1029^{263} \pmod{1443}$$
and sends $c = 1320$ to $\mathcal{B}$. In the meanwhile, $\mathcal{B}$ computes d such that $eq \equiv 1 \pmod{\lambda}$, in our case $d = 671$. Hence, upon reciving $c, \mathcal{B}$ dechyphers it as
$$c^d \pmod{n} = 1320^{671} \pmod{1368} = 1029.$$

■

- 3.3 - The ElGamal algorithm.

After Diffie and Hellman published their paper, apart from the RSA protocol, a new scheme was proposed: the ElGamal protocol. Formally speaking, the algorithm is based on the Discrete Logarithm Problem (DLP): fixed a group $\mathrm{GF}(p)$ and two non-zero elements g and h, find x such that
$$g^x \equiv h \pmod{p}.$$

We discuss the algorithm as follow: first we describe the algorithm in the case of integers modulo a prime, then we discuss it in the context of elliptic curves.

- 3.3.1 - The algorithm: the modulo p case.

Let a Trusted Third Party (TTP) choose a (large) prime p; consider the field $\mathrm{GF}(p)^*$ and let γ be a unit. That is, let γ be such that $\langle\gamma\rangle = \mathrm{GF}(p)^*$. Tell $\mathcal{A}$ to choose $a \in \{2, \ldots, p-1\}$ and compute

$$\alpha \equiv \gamma^a \pmod{p}.$$

The quantity α is public in the channel.

Let m be a message $\mathcal{B}$ wants to send to $\mathcal{A}$. We can assume m to be an integer in the range $[2, p]$. To chyper $m, \mathcal{B}$ chooses a integer k, computes

$$c_1 \equiv \gamma^k \pmod{p} \quad \text{and} \quad c_2 \equiv m\alpha^k \pmod{p}$$

and send the pair (c_1, c_2) to $\mathcal{A}$.

Observe that $\mathcal{A}$ can easily decode the message; in fact, $\mathcal{A}$ can compute

$$x \equiv c_1^a \pmod{p}$$

hence, x^{-1} is known as well. Once x^{-1} is known, computing $x^{-1}c_2 \pmod{p}$ gives us m. It's easily seen that

$$\begin{aligned} x^{-1}c_2 &\equiv (c_1^a)c_2 \pmod{p} \\ &\equiv (\gamma^{ak})^{-1}m\alpha^k \pmod{p} \\ &\equiv (\gamma^{ak})^{-1}m\gamma^{ak} \pmod{p} \\ &\equiv m \pmod{p}. \end{aligned}$$

Example

Say out TTP chose $p = 47$ and $\gamma = 3$. To compute their public key $\alpha = \gamma^a, \mathcal{A}$ chooses $a = 27$. Hence, the value $\alpha = 34$ is known on the channel.

Let $m = 45$ be the message $\mathcal{B}$ wants to send to $\mathcal{A}$, and to compute c_1, c_2 they choose $k = 11$. Then, we have

$$c_1 \equiv 4 \pmod{p} \qquad c_2 \equiv 5 \pmod{p}.$$

Upon recieving the pair (c_1, c_2) can easely decode m. Observe that, $x \equiv c_1^a \equiv 21 \pmod{47}$ and it can be easely checked that $x^{-1} = 9$. Therefore, it follows

$$\begin{aligned} x^{-1}c_2 &\equiv \\ &\equiv c_1^a c_2 \\ &\vdots \\ &\equiv 45 \pmod{47} \end{aligned}$$

which is m as expected. ■

- 3.3.2 - The algorithm: the elliptic curve case.

A trickier implementation of the ElGamal algorithm makes use of elliptic curves. Recall that the "sum" of two points P and Q of some elliptic curve $\mathcal{C}$ is not trivial.

Back to the algorithm: a TTP fixes a elliptic curve $\mathcal{C}$ over the some field $\mathrm{GF}(p)$. Analogously to the previous case, an element $\gamma \in \mathcal{C}$ is fixed and the quantitie $\alpha = a\gamma$ is computed and published over the channel. The remainder of the algorithm is similar to that previously discussed; the main difference is that multiplications are replaced by additions.

Remark. Why do we prefer elliptic curves then? The answer is simple: the computation of $n\gamma$ is expensive.

- 4 - Digital signature.

In todays era of digital documents, it is fundamental to have a way to garantee the ownership of a file, a program or whatsoever. Is in this context that digital signature found its use.

In what follows, we will denote by $\mathcal{S}$ the signature of the document and by $\mathcal{V}$ the trusted entity that proceed to verify the signature; addionally, what we consider is not the document in itself, rather its hash.

- 4.1 - RSA digital signature.

Signing a document by means of the RSA algorithm is somewhat straightforawrd. Let d is the hash value of some document $\mathcal{A}$ produced; to sign d she proceed as follow: she chooses two primes p and q, computes their product $n = pq$ and publish n and a public verification exponent.

Following a similar reasoning to that of *Section 3.2.1*, the congruence

$$sv \equiv 1 \pmod{(p-1)(q-1)}$$

is solved with respect to s. Here s is the *signing exponent* and v the *verification exponent*. Lastly, the document is singned as

$$\mathcal{S} \equiv d^s \pmod{n}.$$

To check the signature, $\mathcal{V}$ simply computes

$$\mathcal{S}^v \pmod{n}$$

and checks whether this is d or not. The correctness of this is given by Euler's formula.

Similarly to the RSA encryption scheme, the RSA signature securety follows from the hardness of computing the v-th root of d, rather then the factorization of n.

- 4.2 - ElGamal digital signature.

Let a TTP choose a large prime p and an integer g such that $\mathrm{GF}(p) = \langle g \rangle$. Say $\mathcal{A}$ has to sign a document d; before doing so they must choose a signing exponent s and compute the quantity

$$v \equiv g^s \pmod{p}.$$

The quantity v, alongside the parameter g and p, represent $\mathcal{A}$'s public verification key.

The signature itself is produced, by choosing an integer $e < p$ and computing the quantities

$$S_1 \equiv g^e \pmod{p} \qquad S_2 \equiv (d - sS_1)e^{-1} \pmod{p-1}.$$

To verify the signature, $\mathcal{V}$ checks that

$$v^{S_1} S_1^{S_2} \pmod{p} = g^d \pmod{p}.$$

Note that the above equation correctly checks the signature; in fact, we have that

$$\begin{aligned} v^{S_1} S_1^{S_2} &\equiv g^{sS_1} g^{eS_2} \\ &\equiv g^{sS_1 + eS_2} \equiv g^{sS_1 + e(d - sS_1)e^{-1}} \equiv g^d \pmod{p}. \end{aligned}$$

- 5 - Integers factorization.

In this section we describe some interesting algorithms to factor integers; additionally, we describe how Euler algorithm to compute the MCD can be improved and how to efficiently compute the modulo of a large integer.

- 5.1 - Pollard's $p-1$ algorithm.

Should be clear by now that prime numbers are at the core of cryptography, since these are hard to find[2]. On the other hand, as in the case of RSA, we need to find distinct prime numbers that are not big, yet not easily computable. Meaning a test for primality would be a good tool. Luckly such tool exists – Pollard's $p-1$ algorithm – though it's applicable to a specific type of numbers.

Let's consider the details. Pollard's algorithm is based on the concept of B-smooth numbers; that is, numbers whose prime factors are all smaller than or equal to a given bound B. Precisely, let $n = pq$, where p, q are prime, be the integer to factorize; assume we have found a integer L such that

$$p-1|L \text{ and } q-1 \not| \, L.$$

This means that there exists i, j and $k, k \neq 0$ such that

$$L = i(p-1) \text{ and } L = j(q-1) + k.$$

By Fermat little theorem we have

$$\begin{aligned} a^L &\equiv 1 \pmod{p} \\ a^L &\equiv a^k \pmod{q}. \end{aligned}$$

Then, we have that

$$p|a^L - 1 \text{ and } q \not| \, a^L - 1,$$

for most values of a. It follows that $p = \mathrm{GCD}\big(a^L - 1, n\big)$.

The interesting part is that, if $p-1$ is such that it's the product of small prime numbers, then it divides $n!$. In other words, what we do is fix $a \in \mathbb{Z}^+$ and compute $\mathrm{GCD}\big(a^{n!}, n\big) = d$; if $d \neq 1$ or $d \neq n$ we have found a non-trivial factor, otherwise we try again by changing the value of a.

Is easily noted that the algorithm has, at least apparently two severe drawbacks: the computation of the factorial and the consequential complexity. Let us say that, how would be clear later in this section, such drawbacks do not exists.

Back to the algorithm: we've said that Pollard's algorithm works if either $p-1$ or $q-1$ are product of small primes; the question is how likely this

[2]The statement is true unless *Riemann* hypothesis turns out to be true.

happens? It's here that B-smoothness plays a key role: if n is B-smooth, then it divides $B!$; additionally, any prime l dividing n has to be less than or equal to B.

- 5.2 - Pollard's ρ algorithm.

We now discuss an algorithm that can be used to solve, when it works, the DLP. Hence, this algorithm is a good tool for cracking ElGamal-based cryptosystems.

The idea is the following: let S be the set of elements we are working with, and consider a function $f : S \to S$ that randomly distributes the elements in S. Since for our purpouse S is finite, by applying f repetedly it will happen that

$$f(x_i) = f(x_j), i \neq j.$$

The only problem is to determine such a function.

In our context of the ElGamal algorithm, let GF(p) be the choosen field and let g be a primitive root. Then, as prosed in ([2]), we can use the function

$$f(x) = \begin{cases} gx, \text{if } 0 \leq x < p/3; \\ x^2, \text{if } p/3 \leq x < 2p/3; \\ ax, \text{if } 2p/3 \leq x < p; \end{cases}$$

where x is reduced modulo p. We can assume that $x_0 = 1$ and by applying f we get simply

$$g^{\alpha_i} a^{\beta_i}$$

where the values of α_i, β_i are given by

$$\alpha_i = \begin{cases} \alpha_{i-1} + 1, \text{if } 0 \leq x < p/3; \\ 2\alpha_{i-1}, \text{if } p/3 \leq x < 2p/3; \\ \alpha_{i-1}, \text{if } 2p/3 \leq x < p; \end{cases}$$

$$\beta_i = \begin{cases} \beta_{i-1}, \text{if } 0 \leq x < p/3; \\ 2\beta_{i-1}, \text{if } p/3 \leq x < 2p/3; \\ \beta_{i-1} + 1, \text{if } 2p/3 \leq x < p. \end{cases} .$$

Then, if we compute $y_i = f(f(y_{i-1}))$, with $y_0 = x_0$ and check when $y_i = x_i$ we get something of the form

$$g^{\alpha_i} a^{\beta_i} = g^{\gamma_i} a^{\delta_i}$$

where γ_i, δ_i are defined by the same relation of α and β. Hence, we have that

$$g^{\alpha_i - \gamma_i} = a^{\beta_i - \delta_i}.$$

Putting $u = \alpha_i - \gamma_i$ and $v = \beta_i - \gamma_i$ and considering the logarithm, we get

$$v \log_g a \equiv u \pmod{(p-1)}.$$

Recall that p is known, therefore, if $\mathrm{GCD}(v, p-1) = 1$ we can multiply by v^{-1} and easely compute the logarithm. Viceversa, if $\mathrm{GCD}(v, p-1) \geq 2$, we can apply the Eucledian algorithm to find s such that $su \equiv d \pmod{(p-1)}$. Multiplying both sides by s, we get

$$d \log_g a \equiv su \pmod{(p-1)},$$

whose solutions is the set

$$\left\{ \frac{w}{d} + k\frac{p-1}{d} \,\middle|\, 0 \leq k < d \right\}.$$

- 5.3 - Lenstra factorization.

Differently from the two algorithms previously described, Lenstra's factorization algorithm assumes we are working with an elliptic curve $C*$ over the ring $\mathbb{Z}/n\mathbb{Z}$. The key idea is that, while computing kP, for some $k \in \mathbb{Z}$ and $P \in \mathcal{C}$, we will eventually have to compute the inverse of a element of the ring that admtis no inverse. Precisely we proceed as follow:

1. Let $\mathcal{C}$ be the elliptic curve $Y^2 = X^3 + AX + B$ over the ring $\mathbb{Z}/n\mathbb{Z}$.
2. Fix a point $P = (a, b)$ at random and $B \equiv b^2 - a^3 - aA \pmod{n}$.
3. For $j = 2, 3, 4, \ldots$ up to some bound do:
   (a) Compute $Q \equiv jP \pmod{N}$ and set $P = Q$.
   (b) If the previous step fails, we have found a non trivial divisor d.
   (c) If $d \leq n$, return d.
   (d) If $d = n$ go back a choose a new point, a new curve or both.
4. Go back to step 2.

Lenstra's factorization algorithm requires $\mathcal{O}\left(e^{\sqrt{(\log n)(\log\log n)}}\right)$ time.

- 5.4 - Extended Euclidean algorithm.

Assume we are interested into computing the GCD d of two integers a and b. From what we have discussed so far, should be obvious by now, that we are also interested into knowing what are the coefficient α, β such that

$$d = \alpha a + \beta b.$$

Hence, the classic Euclidean algorithm cannot be used. Thankfully, there exists an *extended* algorithm that solves the problem. We present below the pseudocode.

Algorithm 1: Extended Euclidean Algorithm

Input: two non-negative Integers a, b with $a \geq b$.
Output: $d = \mathrm{GCD}(a, b)$ and the integers x, y such that $ax + by = d$.

If $b = 0$, return $(a, 1, 0)$.
Set $x_2 \leftarrow 1, x_1 \leftarrow 0, y_2 \leftarrow 0, y_1 \leftarrow 1$.
While $b > 0$ do:
    $q \leftarrow \lfloor a/b \rfloor, a \leftarrow a - qb, x \leftarrow x_2 - qx_1, y \leftarrow y_2 - qx_1$.
    $a \leftarrow b, b \leftarrow r, x_2 \leftarrow x_1 x_1 \leftarrow x, y_2 \leftarrow y_1,$ and $y_1 \leftarrow y$.
Return (a, x_2, y_2).

- 5.5 - The Square and multiply algorithm.

We have discussed so far of how the securety of our algorithms is given by, among other things, the necessifty of computing big numbers reduced in modulo. The question then is, can we ease the task when the user has rights on the data? The affirmative answer is provided by the *Square-and-Multiply* algorithm. *Figure 2* shows the pseudo-code of the algorithm.

Informaly speaking, given a number in exponential form; we decompose the exponent as subsequent powers of two, compute the reduced exponentiations and multiply the results, reducing in modulo accordingly.

Algorithm 2: Square and Multiply algorithm

Input: $g \in \mathbb{Z}_n$ and an integer $0 \leq e < n$, whose binary representation is $e = \sum_{i=0}^{t} e_i 2^i$
Output: $g^e \pmod{n}$

Set $b \leftarrow 1$. If $e = 0$ return b.
Set $A \leftarrow g$.
If $e_0 = 1$, set $b \leftarrow g$.
For i from 1 to t do:
    Set $A \leftarrow A^2 \pmod{n}$.
    If $e_i = 1$, set $b \leftarrow A * b \pmod{n}$.
Return b.

More rigorously, let g be the base number, let e be the exponent and n the modulo. That is, we want to compute

$$g^e \pmod{n}.$$

For the sake of our discussion, assume $e >> 1$.

Essentially, we consider all the subsequent powers of 2, up to $2^{\log e}$. Write e in it's binary expansion and compute g^e as the product of g^{2^i}, where 2^i is in the binary expansion of e.

References.

[1] Whitfield Diffie and Martin E. Hellman. "New directions in cryptography". In: *IEEE Trans. Inf. Theory* 22.6 (1976), pp. 644–654. DOI: 10.1109/TIT.1976.1055638.

[2] J. M. Pollard. "Monte Carlo Methods for Index Computation $\pmod{p}$". In: *Mathematics of Computation* 32.143 (1978), pp. 918–924. DOI: 10.2307/2006496.

[3] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems". In: *Commun. ACM* 21.2 (1978), pp. 120–126. DOI: 10.1145/359340.359342.

[4] Claude E. Shannon. "A mathematical theory of communication". In: *Bell Syst. Tech. J.* 27.3 (1948), pp. 379–423. DOI: 10.1002/J.1538-7305.1948.TB01338.X.

Acronyms.

DLP Discrete Logarithm Problem. 7

GCD Greatest Common Divisor. 2

IFP Integers Factorization Problem. 6

LCM Least Common Multiple. 2

TTP Trusted Third Party. 8